

Parte II — Búsqueda distribuida y condición de parada

Objetivo

El objetivo de esta parte es optimizar la búsqueda distribuida de listas negras para que no sea exhaustiva, es decir, que la ejecución se detenga tan pronto como se detecte el número de ocurrencias necesarias para determinar que un host no es confiable.

En la implementación inicial, cada hilo recorría completamente su rango de servidores asignados y el resultado final solo se evaluaba después de que todos los hilos terminaban su ejecución.

Incluso cuando el host ya estaba claramente clasificado como no confiable, el sistema continuaba consultando servidores innecesariamente.

Esto generaba:

- Uso innecesario de CPU.
- Mayor tiempo de respuesta.
- Una búsqueda completamente exhaustiva, contraria al objetivo del sistema.

Estrategia de solución

Se implementó una condición de parada global, compartida entre todos los hilos, que permite detener la búsqueda tan pronto como se alcance el umbral de ocurrencias definido.

Para esta solución se decidió utilizar un **AtomicInteger compartido**, el cual lleva el conteo global de ocurrencias detectadas por todos los hilos.

Esta elección permite garantizar operaciones atómicas, correcta visibilidad entre hilos y evita el uso de sincronización pesada.

Cambios realizados

- **Clase Host BlackList Validator:** Esta clase es responsable de crear y coordinar los hilos, definir el contador global compartido, recolectar resultados y generar el reporte final.
 - Se creó un **contador compartido** (AtomicInteger) para todos los hilos, el cual representa el número total de servidores que han reportado la IP como maliciosa.

```

public List<Integer> checkHost(String ipAddress, int N)
    throws InterruptedException {

    HostBlacklistsDataSourceFacade facade = HostBlacklistsDataSourceFacade.getInstance();

    int totalServers = facade.getRegisteredServersCount();

    int blockSize = totalServers / N;
    int start = 0;
    AtomicInteger occurrencesCount = new AtomicInteger(initialValue: 0);

```

- El contador se **pasa como parámetro a cada hilo**, garantizando que todos los hilos operen sobre el mismo estado global.

```

for (int i = 0; i < N; i++) {
    int end = (i == N - 1)
        ? totalServers
        : start + blockSize;

    BlackListSearchThread thread = new BlackListSearchThread(start, end, ipAddress, occurrencesCount);

    threads.add(thread);
    start = end;
}

```

- **Clase BlackListSearchThread:** Esta clase es responsable de consultar un rango específico de servidores, reportar coincidencias y respetar la condición de parada global
 - Se incorpora un contador compartido y define el umbral BLACK_LIST_ALARM_COUNT

```

public class BlackListSearchThread extends Thread{
    private final int start;
    private final int end;
    private final String ipAddress;
    private final AtomicInteger occurrencesCount;

    private final HostBlacklistsDataSourceFacade facade =
        HostBlacklistsDataSourceFacade.getInstance();

    private int checkedCount = 0;
    private final List<Integer> occurrences = new ArrayList<>();
    private static final int BLACK_LIST_ALARM_COUNT = 5;

```

- Se implementó una **condición de parada temprana**, donde cada hilo verifica antes de cada iteración si el umbral ya fue alcanzado. En caso afirmativo, el hilo termina inmediatamente su ejecución sin seguir consultando servidores.

```
@Override
public void run(){
    for (int i = start; i < end; i++) {
        if(occurrencesCount.get()>=BLACK_LIST_ALARM_COUNT){
            break;
        }
        checkedCount++;
        if(facade.isInBlackListServer(i, ipaddress)){
            occurrences.add(i);
            occurrencesCount.incrementAndGet();
        }
    }
}
```

El incremento del contador se realiza mediante una **operación atómica**, lo que garantiza:

- Ausencia de condiciones de carrera.
- Coherencia del valor compartido.
- Correcta visibilidad entre hilos.